

12306 逐字句深挖面试题库（简历原句逐条追问版）

下面按你简历里的原句逐条追问，模拟面试官连续深挖场景。重点覆盖架构取舍、实现细节、数据口径与边界条件，避免“只会背术语”。



模块零：破冰、项目真实性与日常沟通（抗压测试）

1. **基本盘问**：毕业届次、专业基础课（比如操作系统、计算机网络）掌握得怎样？实习时长及最快到岗时间？
2. **项目真实性拷问**：你简历里这两个 12306 项目，是借鉴的开源项目、培训班项目（比如马哥/尚硅谷的），还是你自己从头敲代码做出来的？你具体负责了哪几块？
3. **日常与抗压探底**：老家在哪？心仪的 base 地是哪里？因为什么选开发这条路？平时有什么爱好？运动坚持了多久？（常见于综合面沟通题）



项目一：12306 高并发售票核心网关（后端架构开发）

 简历原句 1：“模拟 12306 售票场景下的高性能微服务网关。针对瞬时爆发流量，重点攻克...保障了万级并发压测下的系统稳定与数据零偏差。”

1. **网络与架构**：你提到的高性能微服务网关，是基于 Spring Cloud Gateway 还是基于 Netty 自己手写的网关？从用户浏览器发送请求，经过网关再到业务服务的全生命周期路由转发链路是怎样的？
2. **压测细节**：这里的“万级并发压测”指并发线程数还是稳定 QPS？峰值/稳态分别是多少？你是怎么压出来的，用了什么工具（JMeter/LoadRunner）？压测机器规格（几核几G）是多少？
3. **对账证明**：“数据零偏差”你是如何得出结论的？是有对账脚本（含口径与公式）还是主要基于日志审计？统计窗口和样本范围是什么？

 简历原句 2：“基于 Cache Aside 模式构建 Caffeine + Redis 二级缓存架构以降低数据库 QPS 指标；引入 Redisson 布隆过滤器拦截非法车次请求，防止缓存穿透；利用 RLock 结合 DCL 双重检查锁解决冷启动下的并发击穿问题。”

1. **二级缓存配合**：Caffeine 是应用内缓存，Redis 是分布式缓存。一波流量打过来，这两层缓存的查询先后顺序是什么？如果 Redis 和 MySQL 数据产生了不一致，你怎么兜底收敛？降低数据库 QPS 具体降了多少（是否有 A/B 实测）？
2. **布隆过滤器穿透**：布隆过滤器的拦截逻辑具体加在代码的哪个环节？布隆过滤器有误判率（假阳性）时会放过一部分非法请求，你如何兜底？如果是临时新增车次，如何保证不被误拦（初始化与增量更新怎么做）？
3. **DCL 击穿防御**：“RLock + DCL 双重检查锁”在代码层面到底是怎么写的，什么叫**双重检查**？去掉第一重或第二重检查分别会引发什么问题？冷启动时锁 Key 粒度怎么设计？

 简历原句 3：“应用 Redis Lua 脚本实现库存原子扣减，规避数据库行锁性能瓶颈；设计分段库存 (Segmented Stock) 多 Hash 节点分摊热点流量；引入 @Transactional 保障退票写入库与 MySQL/Redis 双端库存回补的一致性；构建临界区并发碰撞单测，并在全链路压测中保障零超卖。”

1. **Lua与底仓**：为什么非要在 Redis 里面写 Lua 脚本扣减，如果纯用 Redis 的 Pipeline 或者 `decr` 命令不行吗？数据库行锁会有多大的性能瓶颈？
2. **分段方案死抠**：什么是分段库存多 Hash 节点？假设原本“G101”的 100 张票被分成了 10 个 Hash 槽 (Bucket)，每个槽 10 张票，如果有个用户刚好随机访问到第 5 个槽并且没票了，但其实第 6 个槽还有票，他会被直接误判为“无票”吗？
3. **@Transactional底层**：退票补偿加了 `@Transactional`，它能回滚 Redis 吗？如果不能，回补策略是什么？`@Transactional` 失效的典型场景有哪些？
4. **单测实施**：这句里的“**临界区并发碰撞单测**”，你在代码层面如何构建并发冲撞 (CountDownLatch 还是 CyclicBarrier)？它验证的是“算法级不超卖”还是“全链路不超卖”？

🔍 简历原句 4：“采用 RocketMQ 事务消息实现业务解耦；基于自定义 TransactionListener 串联 Redis 预扣减与订单落库，并提供宕机补偿回补与本地事务回查兜底机制。”

1. **事务消息三阶段**：详细说说 RocketMQ 事务消息是怎么做三阶段提交的 (Half消息怎么发的，何时 Commit/Rollback)？
2. **本地回查与残余清理**：当出现极端网络故障，Broker 收不到提交状态而触发 `checkLocalTransaction`。如果查库一直超时或返回 UNKNOWN，如何避免事务长期悬挂？死信订单如何清理与补偿？

🔍 简历原句 5：“基于 Sentinel 实现核心接口的限流与超时熔断；实现全局统一返回值与异常捕获网关；构建 Daemon 异步线程池替代裸线程，避免并发下的内存溢出风险。”

1. **限流配置**：你的 Sentinel 限流是加在抢票方法上的还是整体接口上的？用了滑动窗口还是令牌桶算法？
2. **全局异常怎么抓**：对于底层抛出的 `SQLException` 或空指针，你如何在入口层统一捕获并封装为 JSON 返回 (例如 `@RestControllerAdvice`)？
3. **线程参数与OOM**：为什么叫 Daemon (守护进程) 异步线程池？裸线程 (`new Thread()`) 并发怎么就引起内存溢出了？你的线程池 `corePoolSize` 和任务队列究竟配了多少？

🤖 项目二：12306 智能数字员工 (Reactive / RAG)

🔍 简历原句 6：“基于 12306 业务构建的响应式多智能体协作系统，具备万级并发下的流式查票、规章问答与防攻击能力。”

1. **系统全貌**：为什么不用单体 MVC 而是要做响应式系统？从前端发起问答到送进大模型，整个项目的核心业务数据流行进路线是怎样的？
2. **防攻击深挖**：简历里的“防攻击”具体指什么？是网络层刷量，还是大模型提示词注入 (Prompt Injection)？分别如何防护？

🔍 简历原句 7：“基于 AIServices 与 Query Router 构建多专家协作体系；实施 Session 前缀分区隔离与实体桥接 (Entity Bridging) 方案，有效解决多 Agent 协作中的历史记忆污染，实现复杂购票指令的‘隔而不离’与准确执行。”

1. **专家联调**：一个庞大复杂的 System Prompt 解决不了复杂场景吗？为什么要搞“多专家协作”和 Query Router？
2. **记忆污染**：多 Agent 协作为什么会产生“历史记忆污染”？你如何识别并量化这个问题？
3. **Session 隔离**：Session 隔离时是串行流转还是借用什么方案存储状态的？前驱的闲聊专家是怎么通过 ** “实体桥接” ** 把必要的车票实体信息安全带给购票专家的？

🔍 简历原句 8：“基于 Spring WebFlux + Project Reactor 重构 AI 交互链路...利用 Sinks.Many 技术实现非阻塞、背压可控的实时流式推送；集成 Redis 分布式语义缓存降低了重复请求成本。”

1. **流式原理**：跟大模型的交互，如果它生成的字符是非常巨大的，到前端是**切块下发**还是一次性等待结束下发（SSE 原理）？相比 Spring MVC，非阻塞流为什么能抗极高并发？
2. **背压防爆**：当模型产出速度远高于客户端消费速度时会发生什么？**Sinks.Many** 如何配合背压策略避免内存堆积？
3. **语义缓存拦截**：如何判断两个问题语义等价？如果异常文本（如限流报错）被写入缓存，如何避免污染后续命中？

🔍 简历原句 9：“针对长文本规章采用 Parent-Child 双层切片策略保障检索连贯性；集成 Milvus 与原生 BM25 双路召回，应用 RRF 算法与 Cross-Encoder 重排模型解决大模型业务幻觉痛点。”

1. **核心幻觉治理**：如何发现并解决大模型在业务当中的幻觉问题？项目里具体怎么设计**提示词卡边界**去减少幻觉？
2. **切片机制**：对于长文本为什么不传整段而是 Parent-Child 切片？如果我问关于整本规章的汇总，它是单独提取全章节还是逐节解决的？
3. **排位机制与大模型选型**：Milvus 与 BM25 打分口径不同，如何融合（RRF 公式或思想）？选择 GLM-4 的依据是什么（效果、成本、时延、稳定性）？后续还在探索哪些能力扩展？

🔍 简历原句 10：“实现‘短期滑动窗口 + 长期摘要压缩’双层记忆架构，防丢且突破 Token 限制；引入 RAGAS 评估框架，利用 LLM-as-a-Judge 机制对召回率等核心维度进行量化评估。”

1. **摘要压缩**：当你对用户的长期聊天进行摘要压缩时，万一关键业务参数“订单号 G123”给压缩丢了，后面怎么购票？这个难点怎么解决的？
2. **评估体系落地**：在 **LLM-as-a-Judge** 方案下，RAGAS 的 **Context Recall** 与 **Faithfulness** 如何定义和打分？你如何基于分数反向优化检索与生成链路？

🔥 模块三：精选高频攻击题（源自常态化八股与架构补充）

根据另一份面试宝典提取，重点补充特定专有名词和边缘攻防场景。

1. 分布式与组件底盘

- **AQS进阶**：你提到 ReentrantLock，能详细讲讲 AQS 底层那个队列具体是什么样子的吗（它的 CLH 变体队列设计）？
- **CAP权衡**：CAP 理论下，购票场景如何在 AP 与 CP 之间权衡？“最终一致性”具体一致的对象是什么？
- **布隆特性的绕过**：既然原生的布隆过滤器不支持动态删除，如果你不想深夜停机全量重构，有没有增加“逻辑布隆位（标记位）”或者附加特定缓存逻辑的规避重构方法？

2. 核心网关细节追击

- **Lua脚本崩溃**：如果 Lua 扣减脚本执行异常，Redis 是否会出现“执行到一半”的中间状态？为什么？
- **Sentinel 算法底层**：你用的 Sentinel 说是滑动窗口限流，能讲一下 Sentinel 底层 **LeapArray** 核心数据结构是如何实现滑动窗口的时间切片流转的吗？比固定窗口好处在哪？

3. 智能 Agent 深度补充

- **意图与编排设计**：为什么你们 AI 系统要设计成 **Orchestrator-Worker**（路由员与执行员分离）编排架构？把意图识别 Agent 和具体执行购票任务的 Agent 分开有什么优势？
- **流式长连接瓶颈**：使用了 WebFlux 实现流式推送（SSE）。如果是用传统的 MVC 来做 SSE，会占用死 Servlet 的工作线程从而拖垮系统吗？WebFlux 在线程调配上具体做了什么改进？
- **语义缓存极值**：你在做 Redis 语义向量缓存（Semantic Cache）时，相似度阈值为什么定在 0.95？如果定在一个更低的数值（比如 0.8），会在你们的抢票/退改签系统中发生什么样的严重事故？